

# 1) Extra functionality ideas that are genuinely useful for an NMR lab

Below are features that tend to remove “Excel pain” for an NMR facility like the Swedish NMR Centre (at University of Gothenburg) and make the GUI feel like a lab tool rather than a customer tool.

## A. Fast workflow controls (the “80 samples” problem)

- **Batch-level bulk actions** (one click):  
“Mark all samples received”, “Mark all prepared”, “Mark all loaded”, “Mark all experiments queued”, “Mark selected as failed”, etc.
- **Row selection + apply action**: checkbox column + “Apply status to selected”.
- **Status auto-rules (optional)**:  
If tray position is set  $\Rightarrow$  sample status becomes “Loaded”. If experiment-id is set  $\Rightarrow$  experiment status becomes “Planned/Queued”. If data found on disk  $\Rightarrow$  “Finished”.
- **Keyboard-centric data entry**: Enter/Tab behavior, arrow navigation, paste support (you already have paste-down), plus “fill down” and “increment numbers” (e.g., start at 10 then 11,12,...).

## B. Reduce mistakes (labs care a lot about this)

- **Validation & conflict warnings**
  - Duplicate tray positions within a batch (highlight both).
  - Missing required metadata (sample type/solvent, experiment type, etc.).
  - “Experiment number already used” (if scanning the folder structure).
- **Locking / freezing**  
When a batch is running, optionally lock columns (prevent accidental edits).
- **Audit log (lightweight)**  
Store “who changed what when” (even if “operator initials” only). Labs love traceability.

## C. Rack/Tray visualization (big adoption lever)

- **Rack map view**: show positions as a grid (SampleJet / carousel / 5mm rack style). Click a position to jump to the sample row.
- **Auto-assign tray positions**: choose rack type + start position + skip blanks  $\rightarrow$  fill tray positions for all samples.
- **Print-friendly “bench sheet”**: one-page summary with tray map + sample labels for tube prep.

## D. Make it multi-customer for the lab (without making it complex)

- **Customer selector + batch list**: the lab chooses customer + batch from a dropdown/search.
- **Per-customer templates**
  - Default experiment types (e.g., 1D 1H + JRES for customer A)
  - Required columns (some customers need concentration, solvent, pH, etc.)
- **Read-only customer view link** (optional): customers can see statuses + download links, but cannot edit.

## E. Instrument / Bruker integration helpers (practical, not “enterprise”)

- “Copy experiment path” action (from the spectrometer PC) + quick instructions to open the path in the local file manager (no app installation).
- **Scan for newly created experiment numbers**  
Detect numeric folders (10, 11, 12...) and offer to assign them to rows (or at least show “unassigned experiments found”).
- **Detect experiment completion**  
Check for fid and/or pdata/1/\* and/or timestamps in audita.txt to set status.
- **Estimated run-time / queue insight** (optional but persuasive)  
Pull rough acquisition time from acquis and show “batch ETA” and “remaining experiments”.

## F. Data delivery that doesn’t involve manual zipping

- **Upload “package” builder**
  - Select batch → select which experiments/types to include → click Upload
  - Auto-create: batch\_manifest.json + optional tracker\_export.xlsx + optionally compress data
- **Retry + partial upload handling** (lab reality)
  - If upload fails mid-way, show what succeeded and allow “retry failed only”
- **Integrity checks**  
Optional checksums (md5/sha256) in manifest.

## 2) Sprint plan

This plan assumes the existing repo:

- Backend: tracker-app/src/app.py (FastAPI + Postgres)
- Frontend: tracker-app/static/index.html (plain JS)
- DB schema: postgres/scripts/schema.sql

## Sprint 0 — Groundwork & refactor

### Goal

Make future changes safer and easier.

### Tasks

#### 1 . Move the inline JS out of index.html

- Create tracker-app/static/app.js
- Keep index.html mostly markup + `<script src="/app.js"></script>`

#### 2 . Add a simple config panel in the UI

- Display “mode”: bench / spectrometer
- Store local UI prefs in localStorage (not DB yet)

#### 3 . Add a minimal backend health + version endpoint

- GET /api/version returns git-ish version string or static version.

### Acceptance criteria

- UI works exactly like today.
- No logic changes yet—only structure.

## Sprint 1 — Status model + bulk status setting (core adoption feature)

## Goal

Add statuses for samples and experiments + fast ways to set them.

## Data model changes (DB)

Update postgres/scripts/schema.sql:

- Add to "sampleExperimentBatches":
  - sampleStatus TEXT (Expected/Received/Prepared/Loaded/Done/Problem)
  - preparedAt TEXT (or TIMESTAMP later)
  - receivedAt TEXT
- Add to "sampleExperiments":
  - experimentStatus TEXT (Planned/Queued/Running/Finished/Failed/Uploaded)
  - startedAt TEXT, finishedAt TEXT
  - errorMessage TEXT

## Backend API

In tracker-app/src/app.py:

1. Add endpoints:
  - PATCH /api/sampleExperimentBatches/{batchId}/status (bulk)
    - body: { sampleIds: [...], status: "Received" }
  - PATCH /api/sampleExperiments/{batchId}/type/{typeId}/status (bulk)
    - body: { sampleIds: [...], status: "Finished" }
2. Ensure GET /api/... includes new columns (already automatic via SQL select).

## Frontend UI

In static/app.js:

- Add a **Status column** per batch (next to Tray Pos / Comment)
- Add **experiment status indicator** in each experiment-id cell (tiny colored dot or border)
- Add **bulk action controls** in the batch header:
  - dropdown: "Set all sample statuses to..."
  - dropdown: "Set all experiment statuses to..."

## Acceptance criteria

- A technician can mark all 80 samples "Received" in 2 clicks.
- Status persists in DB and shows after refresh.
- No breaking changes to existing tray position / experiment-id editing.

# Sprint 2 — Speed tools: auto-fill + validation + rack map

## Goal

Make the GUI faster than Excel and reduce common errors.

## Features

### 1 . Auto-fill tray positions

- Input: rack type (simple: sequential numbers) + start value
- Button: “Assign tray positions”

### 2 . Auto-fill experiment numbers

- Per experiment type: start number + increment

### 3 . Validation warnings

- Duplicate tray positions highlight
- Non-numeric experiment ids (optional rule)

### 4 . Rack map view

- Toggle button “Rack map”
- Click a position to highlight the matching row

## Backend

No new endpoints required if you reuse the existing bulk tray-position endpoint:

- POST /api/sampleExperimentBatches/{batchId}/bulk

## Frontend

- Implement auto-fill functions client-side, then call bulk endpoints.
- Rack map can be generated from current table state (no backend work).

## Acceptance criteria

- Assigning tray positions for 80 samples takes < 30 seconds and no copy-paste.
- Duplicate tray positions are obvious immediately.

# Sprint 3 — Spectrometer browser mode: connect local TopSpin folder

## Goal

Make it genuinely useful on the spectrometer computer using browser-only local folder selection (no installation).

## Data model changes

Add to "experimentBatches":

instrumentName TEXT (optional)

localFolderHint TEXT (optional; display label for the TopSpin batch folder selected in the browser)

Add to "sampleExperiments":

experimentPathHint TEXT (optional; display-only helper such as "10/" or "10/pdata/1")

## Backend API

Add endpoints in app.py:

POST /api/batches/{batchId}/local-scan

Stores derived scan results sent from the browser (numeric experiment folders found + completion flags).

POST /api/spectrometer/{batchId}/sync-status (can also be introduced in Sprint 4)

Accepts scan results and updates experiment statuses in bulk.

## Frontend

In batch header, add:

"Connect local TopSpin folder" button (uses browser folder picker; File System Access API with webkitdirectory fallback)

Display selected folder label + "connected" indicator + last scan time

"Copy experiment path" actions (copy to clipboard) instead of trying to open folders automatically

Optional: show "unassigned experiment folders found" (folders present on disk but not referenced in the table)

## Acceptance criteria

On the spectrometer PC, user can connect the TopSpin batch folder in the browser and see numeric experiment folders detected.

User can copy experiment paths for quick navigation in the local file manager.

# Sprint 4 — Refresh statuses from folder + reconcile completion

## Goal

One button updates the whole table based on what actually ran, using the connected local folder in the browser.

## Frontend

Button: “Refresh status from folder”

Scan the connected folder for each experiment-id and apply completion heuristics:

fid exists and file size > 0  $\Rightarrow$  “Finished/Acquired”

pdata/1/ exists  $\Rightarrow$  “Processed” (optional)

Optionally read audita.txt to extract timestamps (if needed and feasible)

Show a small status summary: “Finished: 62 / 80, Missing: 18”

## Backend

POST /api/spectrometer/{batchId}/sync-status

Updates DB statuses in bulk based on scan results received from the browser.

Optional: store scan history (scannedAt + summary JSON/TEXT) for traceability.

Acceptance criteria

Technician can reconcile the tracker vs reality in one click.

No manual marking of “finished” is necessary (unless the local folder is not connected).

# Sprint 5 — Browser-mediated upload + packaging + manifest

## Goal

Replace manual cloud upload with a controlled, repeatable workflow where the operator uploads selected experiments via the browser (no installation on the spectrometer computer).

## Backend dependencies

Prefer direct-to-object-storage uploads with pre-signed URLs (recommended for large datasets).

Add a server-side library for your chosen storage provider's pre-signing/multipart upload (e.g., S3-compatible), if needed.

## Data model changes

Add a new table "uploadJobs":

id TEXT PRIMARY KEY

experimentBatchId TEXT

createdAt TEXT

status TEXT (Pending/Running/Failed/Done)

log TEXT

remotePath TEXT (download URL or storage path)

Optional: "uploadJobItems" listing which experiment folders were included and total bytes/files.

## Backend API

POST /api/uploads/{batchId}/create

body: { includeExperimentNumbers: ["10","11",...], includeXlsx: true }

POST /api/uploads/{uploadJobId}/presign (recommended)

Returns pre-signed URLs (or multipart upload instructions) for the files the browser will upload.

GET /api/uploads/{uploadJobId} (poll status/log)

POST /api/uploads/{uploadJobId}/complete

Browser posts a manifest.json (list of uploaded experiments/files + sizes + optional checksums) and marks job done.

## Frontend

Batch header:

checklist "upload all finished" / "upload selected"



button “Create upload package” (generates manifest + prepares file list)

button “Upload now” (uploads from browser with progress + retries; uses pre-signed URLs if available)

show progress + result download link/path when complete

### Acceptance criteria

Lab can select experiments and upload without leaving the GUI.

Uploads are resilient for large datasets (progress, retry; optional multipart/resume).

If upload fails, the UI shows failure reason + allows retry.

## Sprint 6 — Multi-customer + templates + read-only view

### Goal

Make the tool something the lab can use for *everyone*, not only QFish.

### Data model changes

Add tables:

- "customers": id, name, defaultExperimentTypes JSON/TEXT, notes
- "orders" (optional): orderId, deliveryId, customerId, createdAt  
Link "experimentBatches" to customerId.

### Backend

- CRUD endpoints for customers + “list batches by customer”
- Endpoint to create a batch from a template:
  - Creates experimentTypesPerBatch entries automatically

### Frontend

- Left pane becomes:
  - customer selector
  - batch selector
  - “new batch” wizard (choose customer → apply template)

Optional read-only:

- A URL parameter ?mode=readonly disabling inputs

### Acceptance criteria

- Lab can manage multiple customers and switch quickly.
- Creating a new batch for a customer auto-adds the correct experiment columns.

# Sprint 7 — Hardening: audit trail, permissions, nicer exports

## Goal

Make it “lab-grade”.

## Features

- **Operator initials** stored with each change (simple text input stored in localStorage + sent to backend)
- **Audit trail table** (append-only):
  - when, who, table, record keys, old/new
- **Export improvements**
  - Include statuses
  - Include “manifest” sheet for uploads
- **Better auth**
  - Keep nginx basic auth, but allow roles later (optional)

## Acceptance criteria

- Changes are traceable.
- Exports contain everything needed for reporting/invoicing and customer communication.

## Notes

- DB schema: postgres/scripts/schema.sql (+ update import/export scripts if you want to persist CSV workflows)
- Backend endpoints: tracker-app/src/app.py
- Frontend: split into tracker-app/static/app.js (create this in Sprint 0)
- Deployment: standard hosted web app. Spectrometer workflows use browser folder selection (no local mounts/agents); large data uploads should use pre-signed URLs/multipart uploads via object storage.

## Highest-impact subset to pitch to the lab

*Pitch this sequence for highest impact:*

1. **Statuses + bulk actions** (Sprint 1)
2. **Rack map + auto-assign** (Sprint 2)
3. Connect local folder + scan + refresh-from-folder (Sprints 3–4)
4. **Upload package** (Sprint 5)
5. **Multi-customer templates** (Sprint 6)

Turns the app from “a QFish tracker” into “the lab’s operating console”.